ApexUCL Tournament Management System

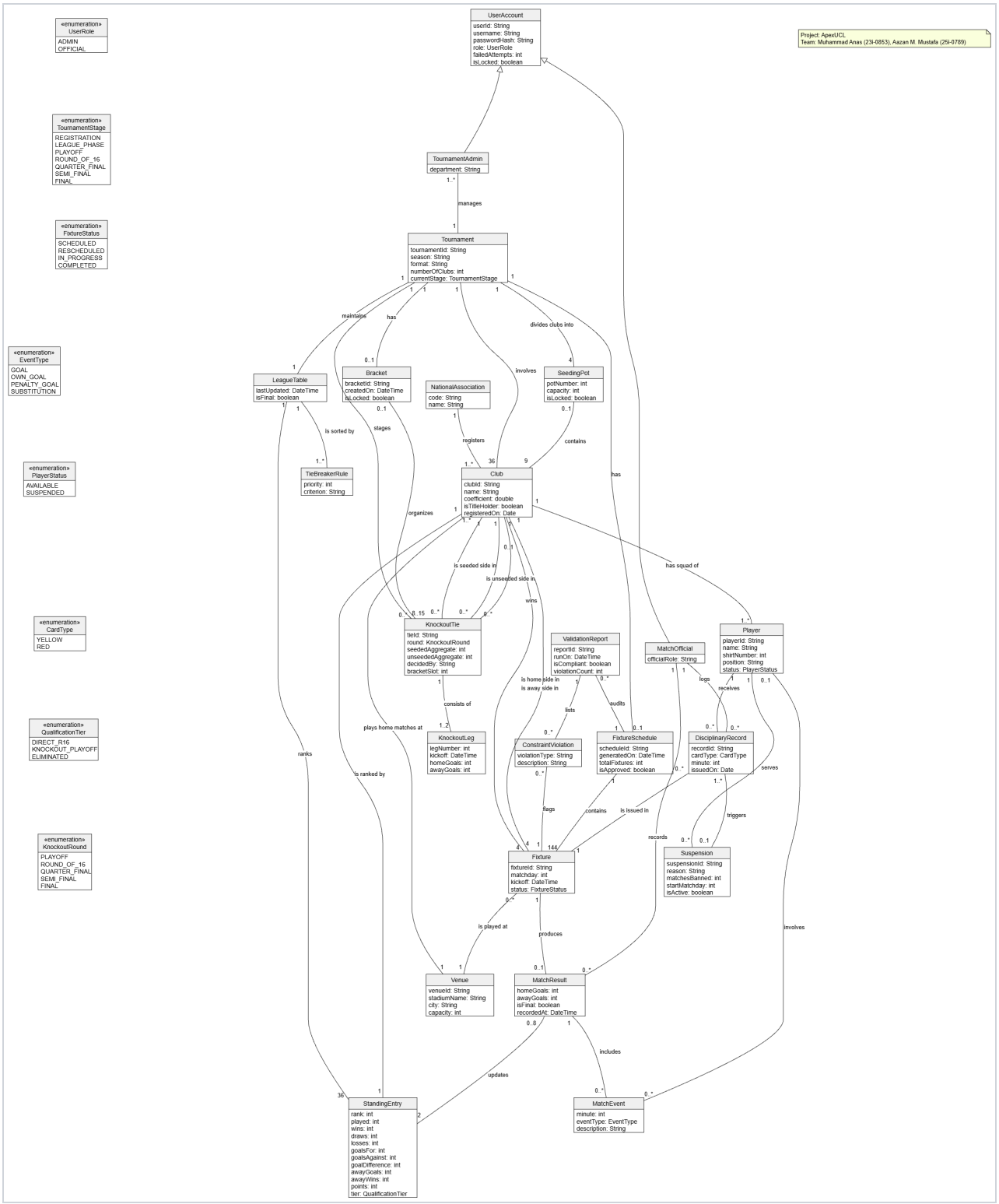
Deliverable 4: Domain Model, System Sequence Diagrams & Code Integration
Course: Software Design & Analysis (SDA)

1. Team Contributions & Role Distribution

Team Member	Roll Number	Assigned Roles	D4 Specific Contributions
Muhammad Anas	23i-0853	Team Manager, Lead Analyst, UI/UX Designer	• Drew all 12 System Sequence Diagrams (UC-01 to UC-12) in Draw.io • Implemented the Employee hierarchy in Java and integrated it into ApexUCL • Testing, final consolidation & submission
Aazan Muhammad Mustafa	25i-0789	Systems Analyst, QA Manager	• Modeled the Domain Model for ApexUCL • Defined conceptual classes, typed attributes, named associations and multiplicities

2. Part One: Domain Model

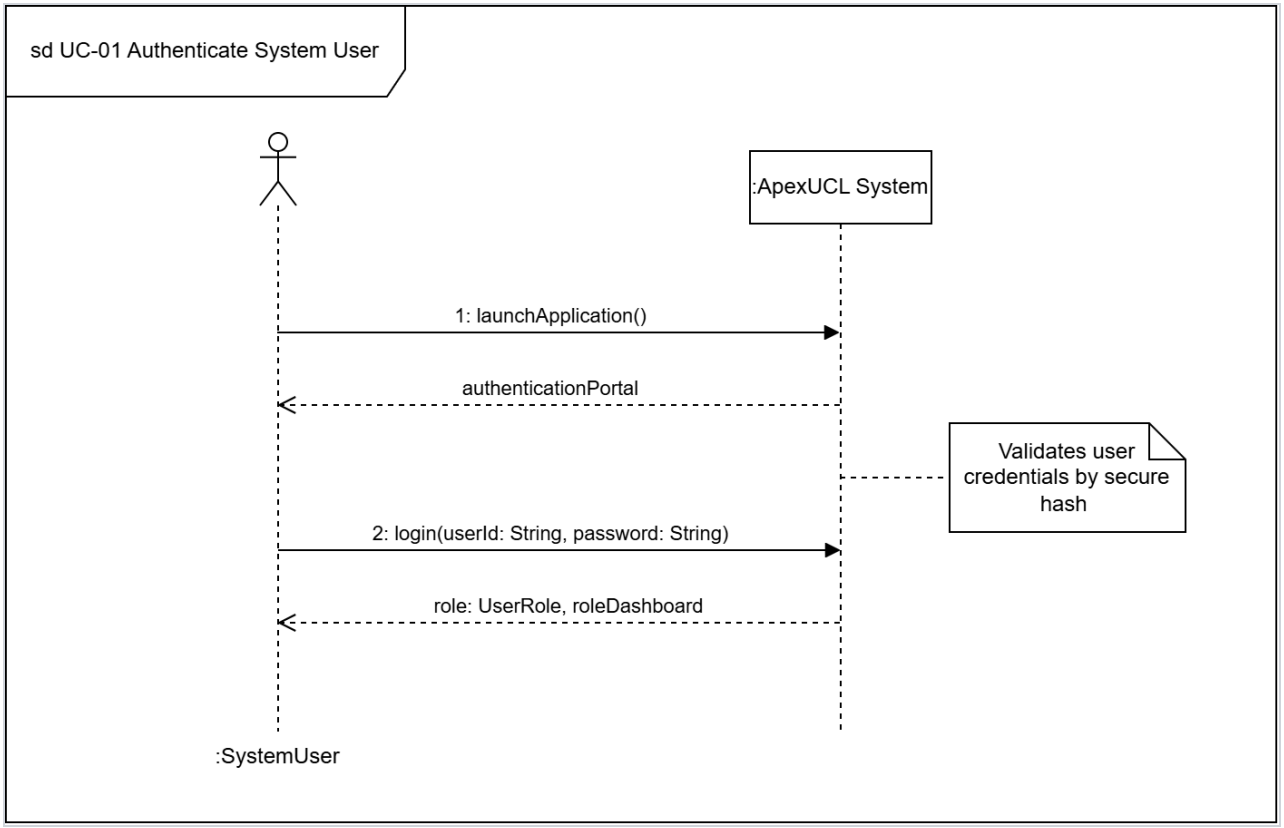
The domain model shows the conceptual classes of ApexUCL with their attributes and data types, and the named associations between them with multiplicity at both ends.



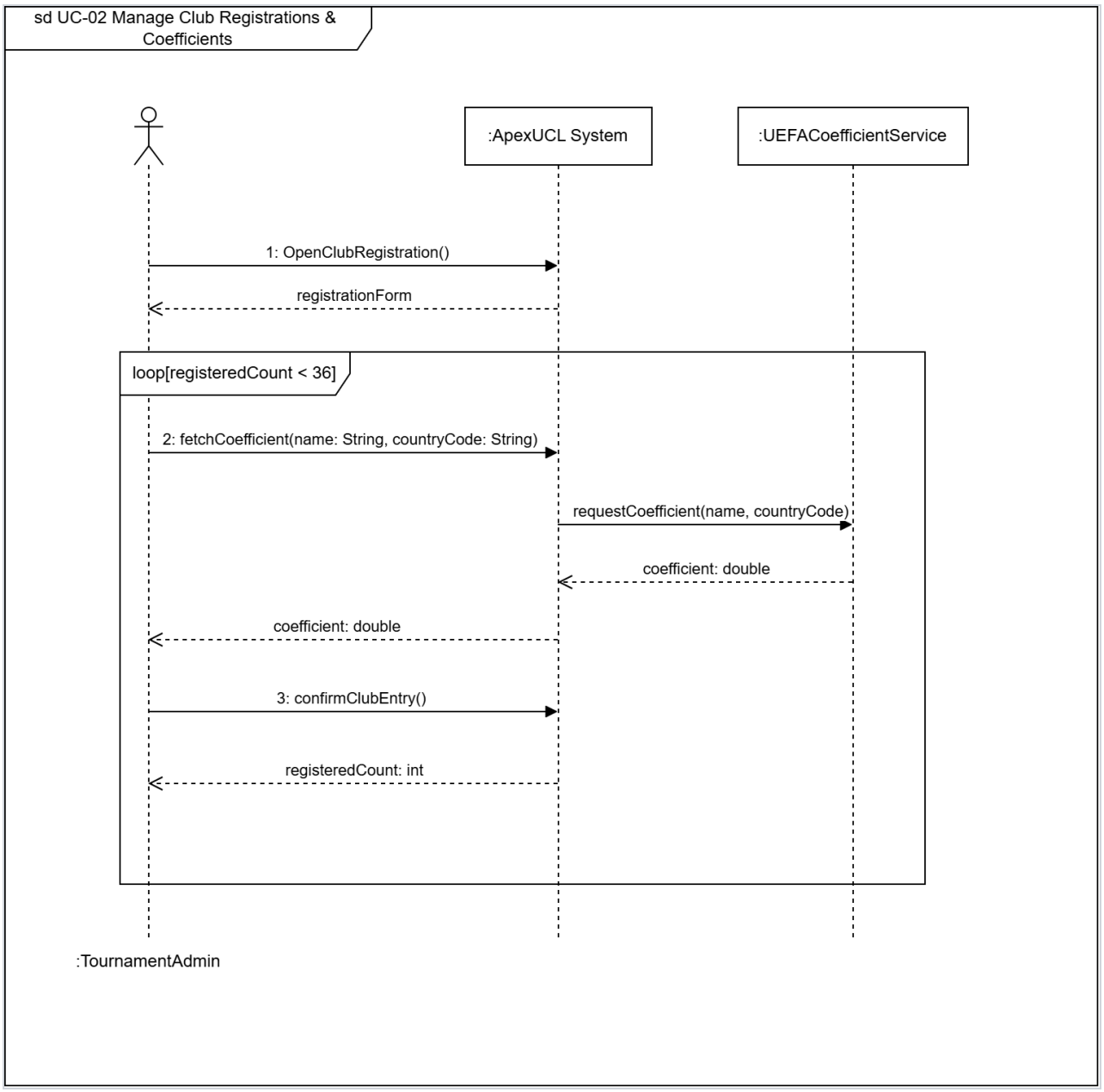
3. Part Two: System Sequence Diagrams

One SSD was drawn in Draw.io for the main success scenario of each of the 12 use cases from Deliverable 3. Each shows the primary actor, the ApexUCL System as a black box, and the system events with their parameters and return values.

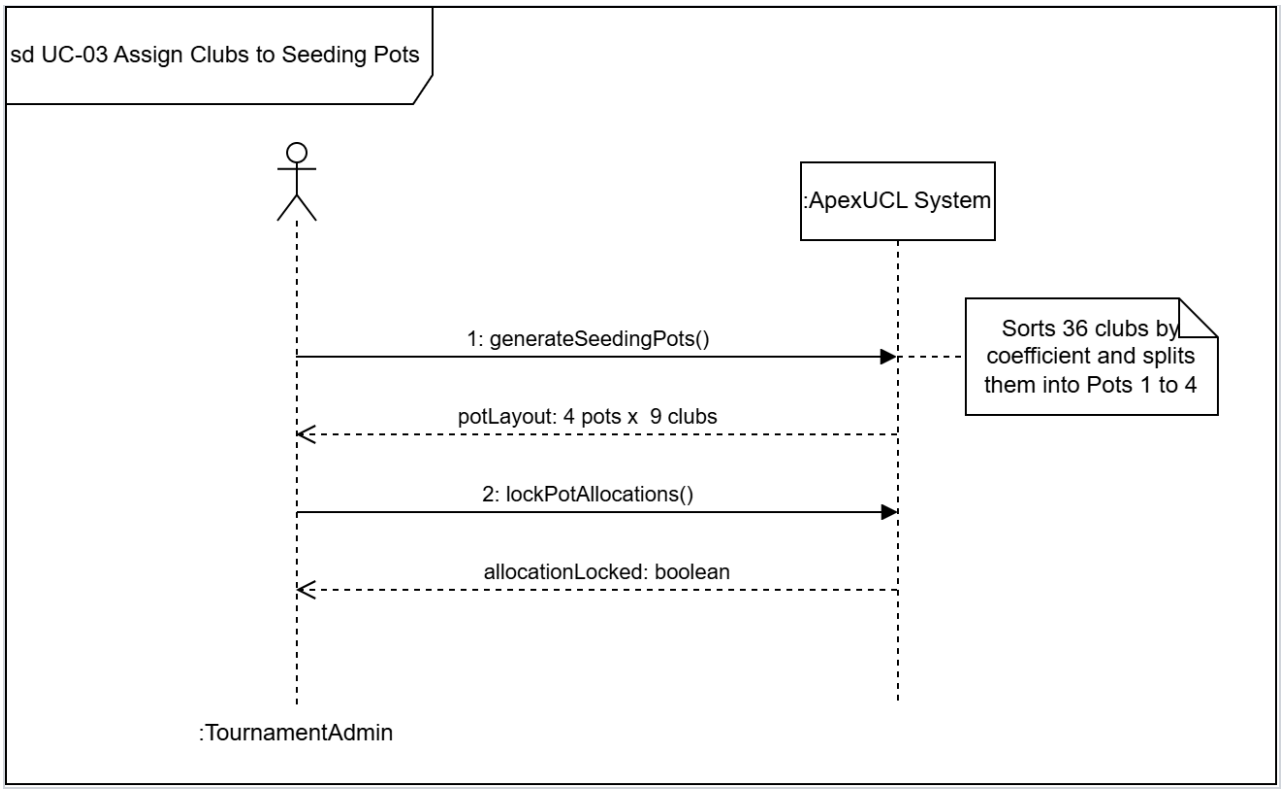
3.1 SSD UC-01: Authenticate System User



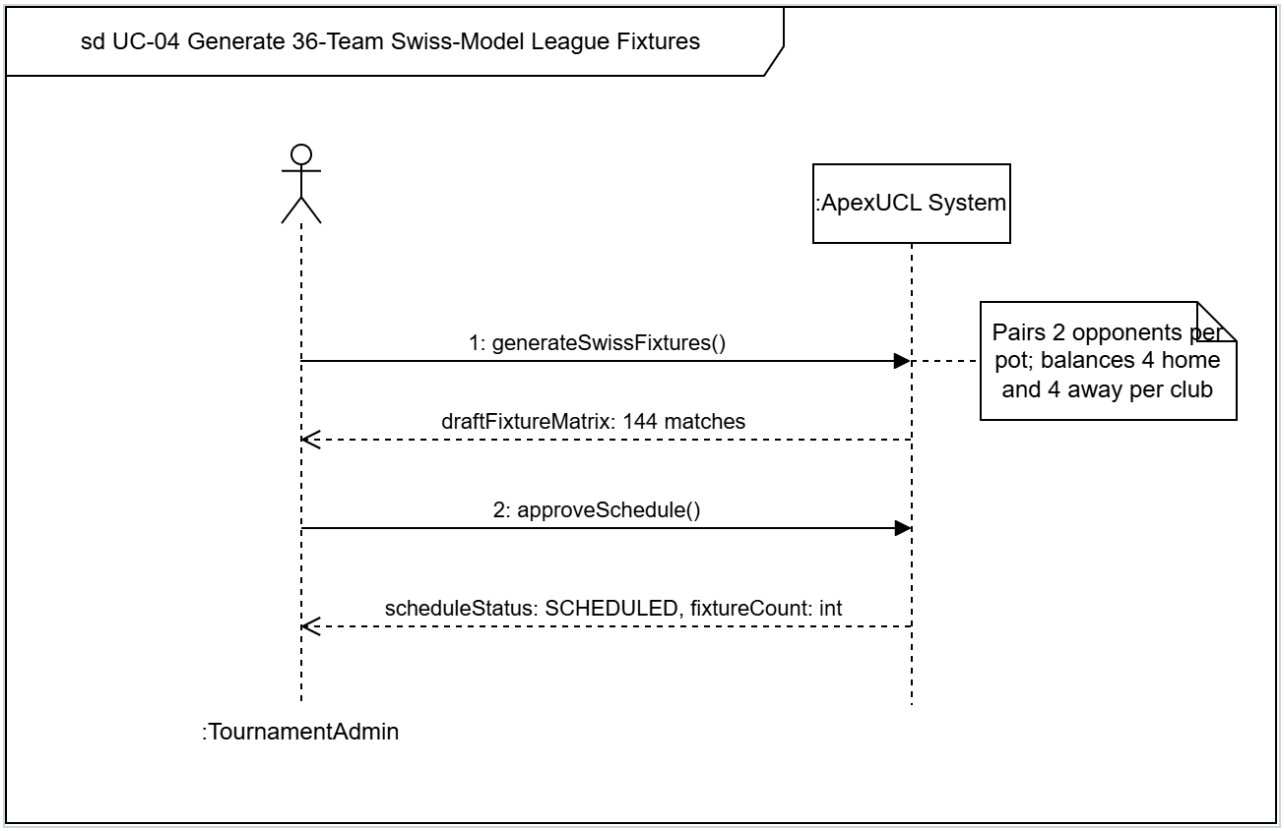
3.2 SSD UC-02: Manage Club Registrations & Coefficients



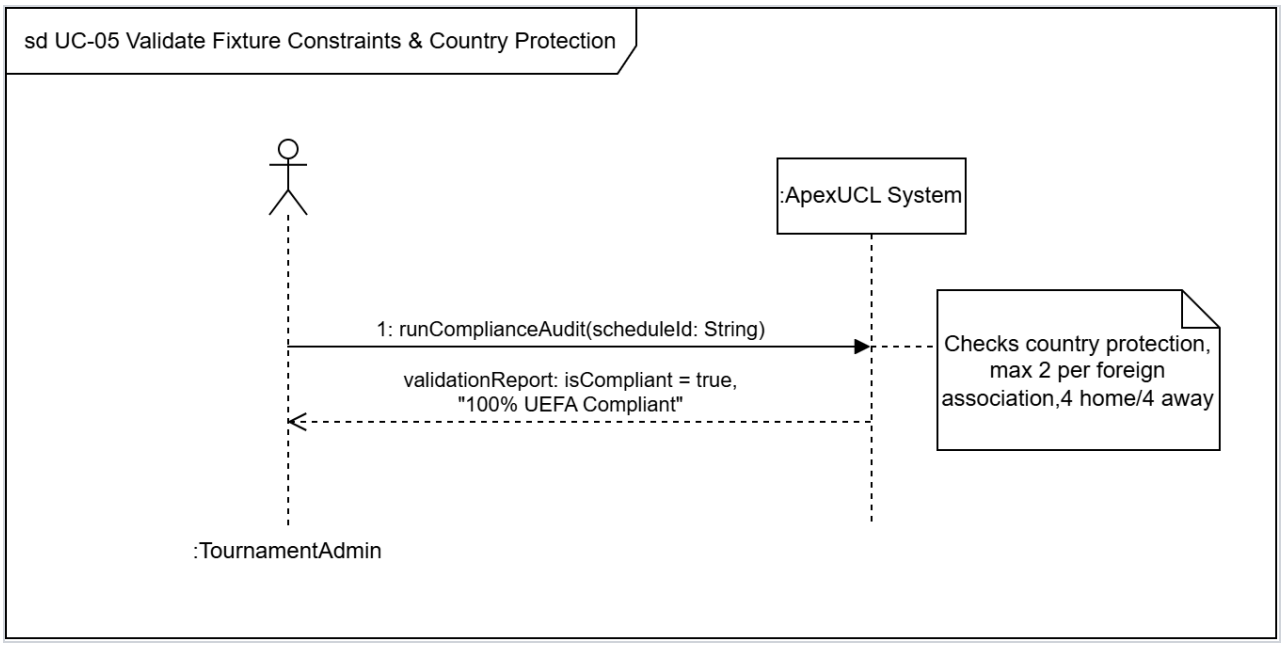
3.3 SSD UC-03: Assign Clubs to Seeding Pots



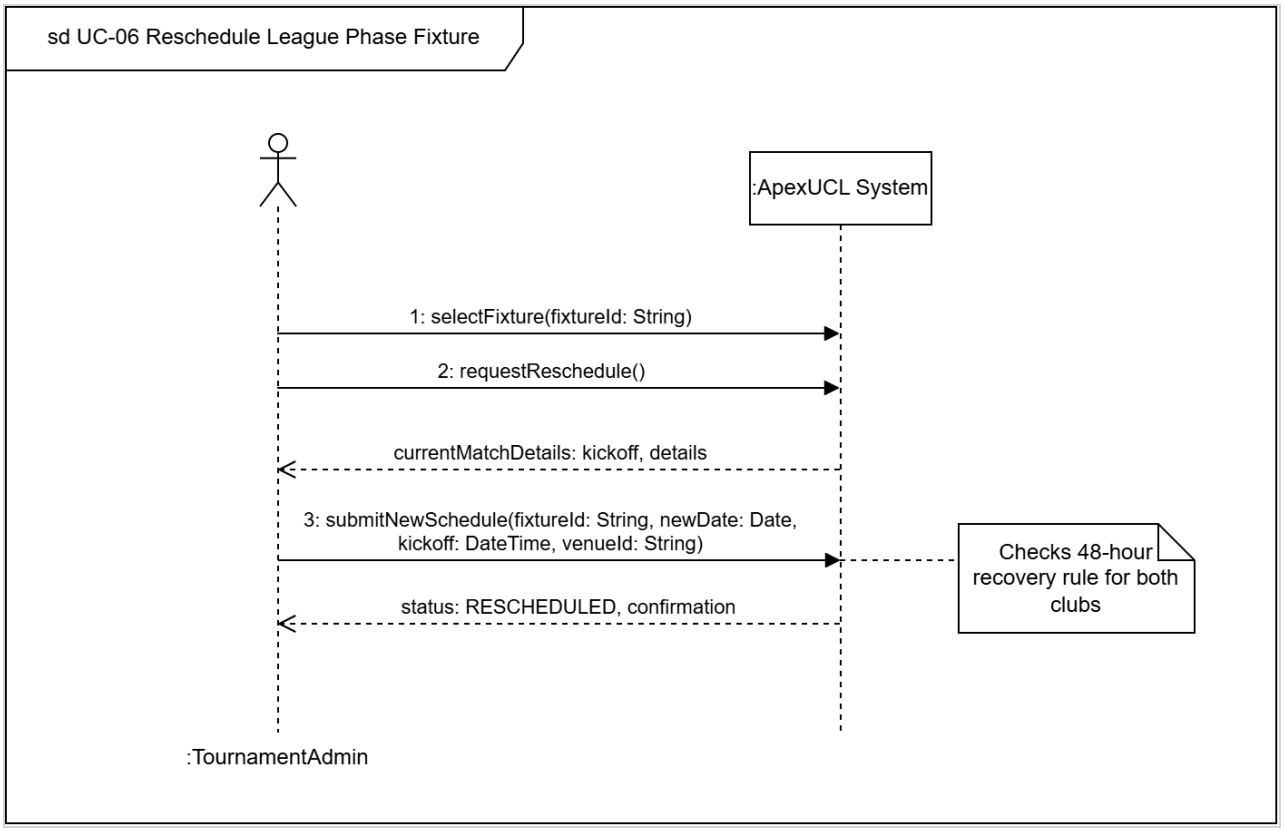
3.4 SSD UC-04: Generate 36-Team Swiss-Model League Fixtures



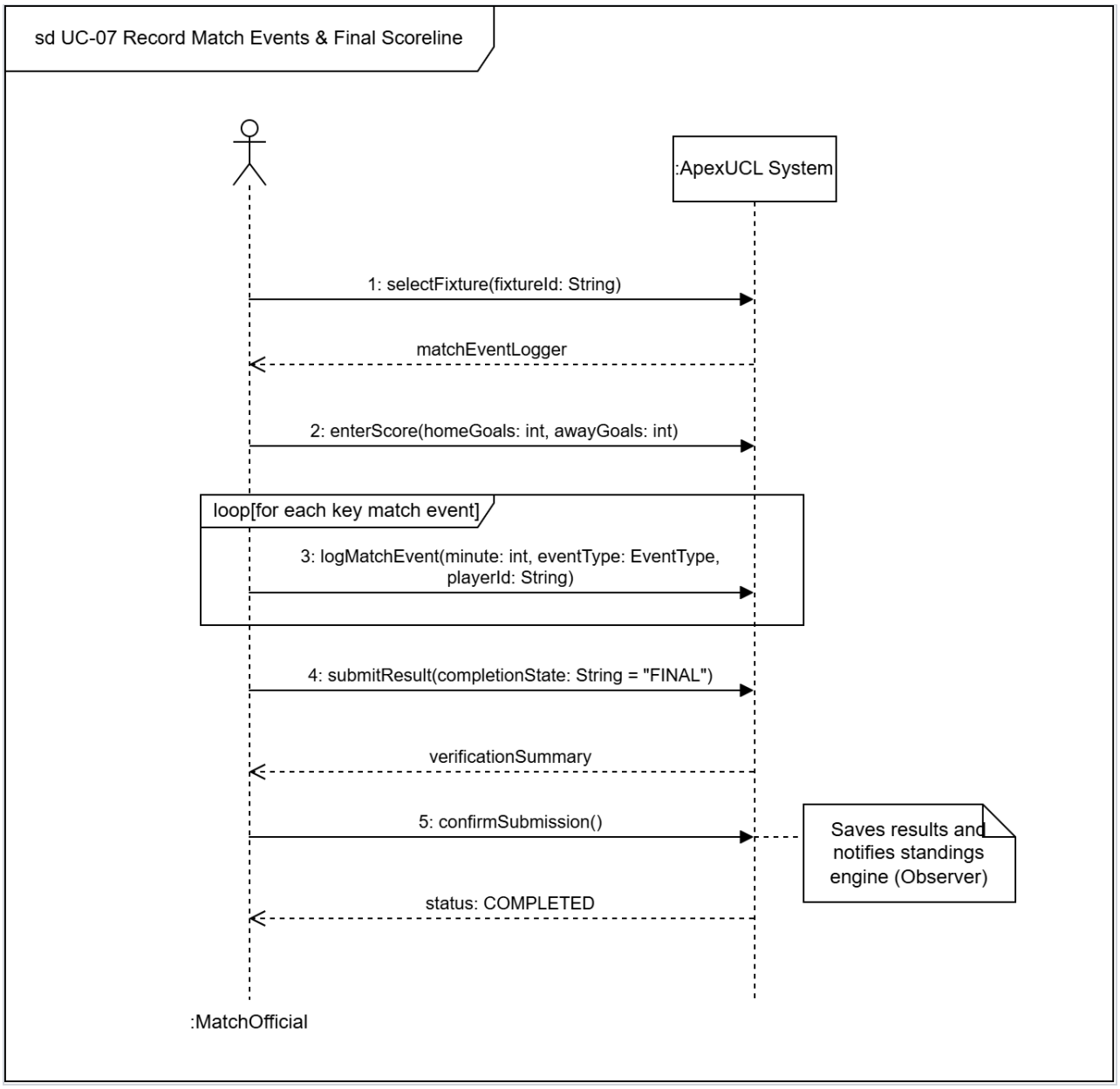
3.5 SSD UC-05: Validate Fixture Constraints & Country Protection



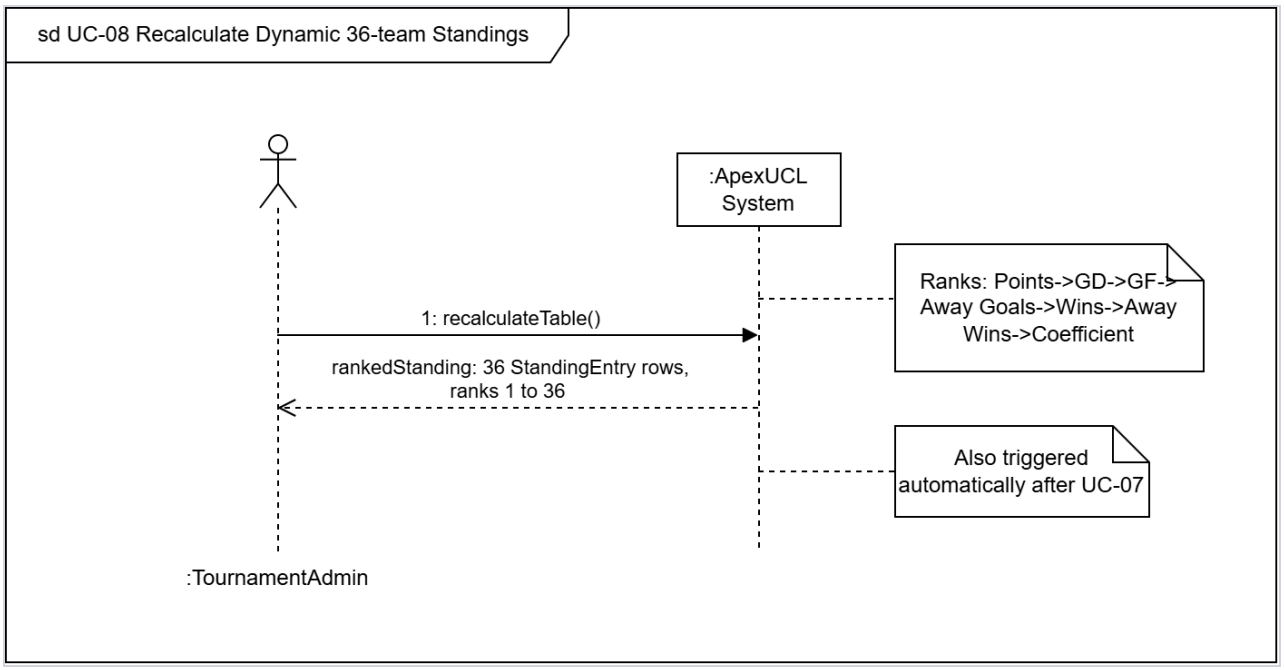
3.6 SSD UC-06: Reschedule League Phase Fixture



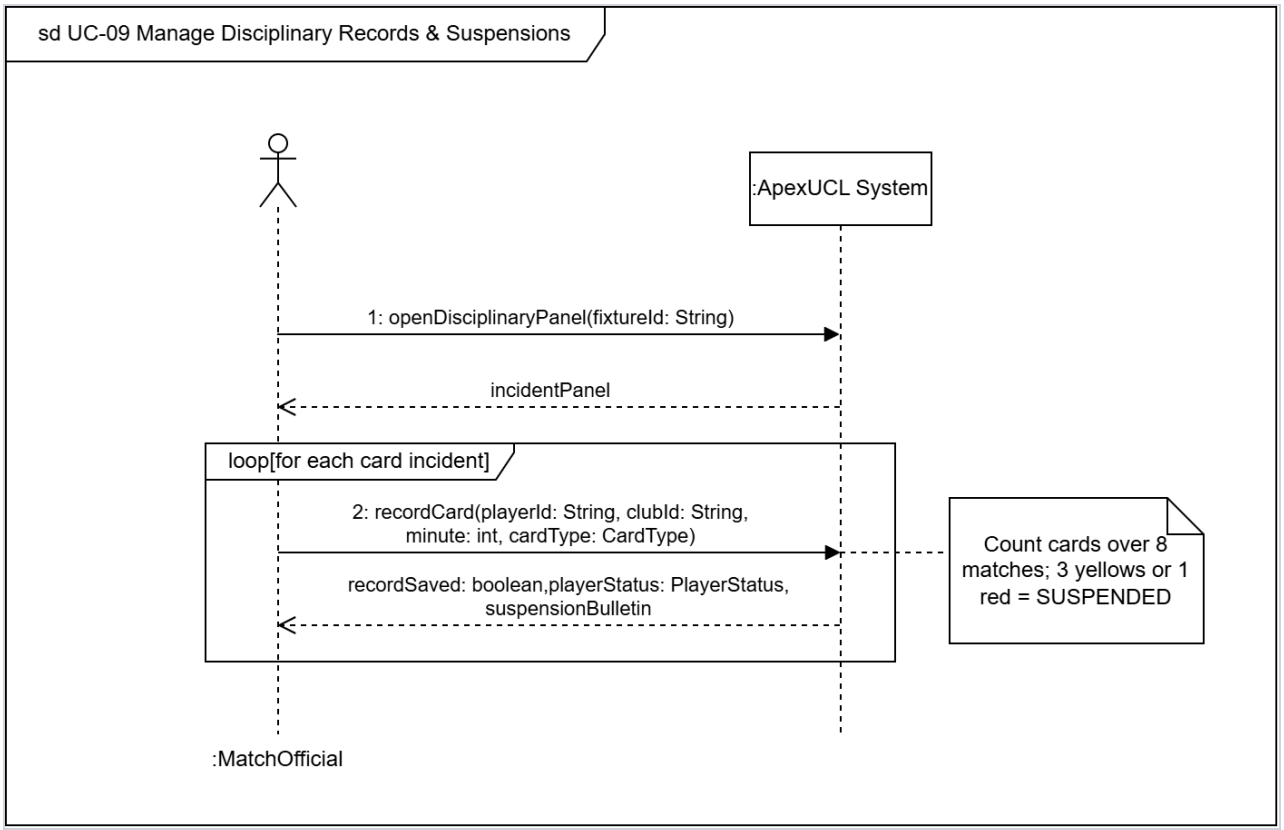
3.7 SSD UC-07: Record Match Events & Final Scoreline



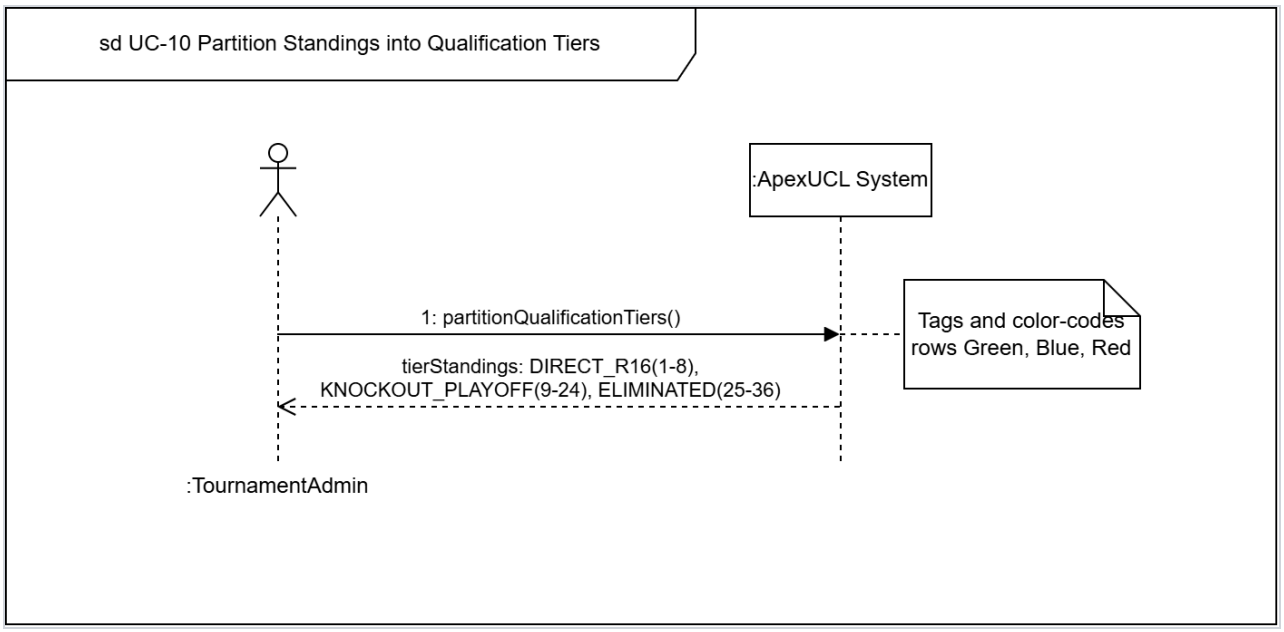
3.8 SSD UC-08: Recalculate Dynamic 36-Team Standings



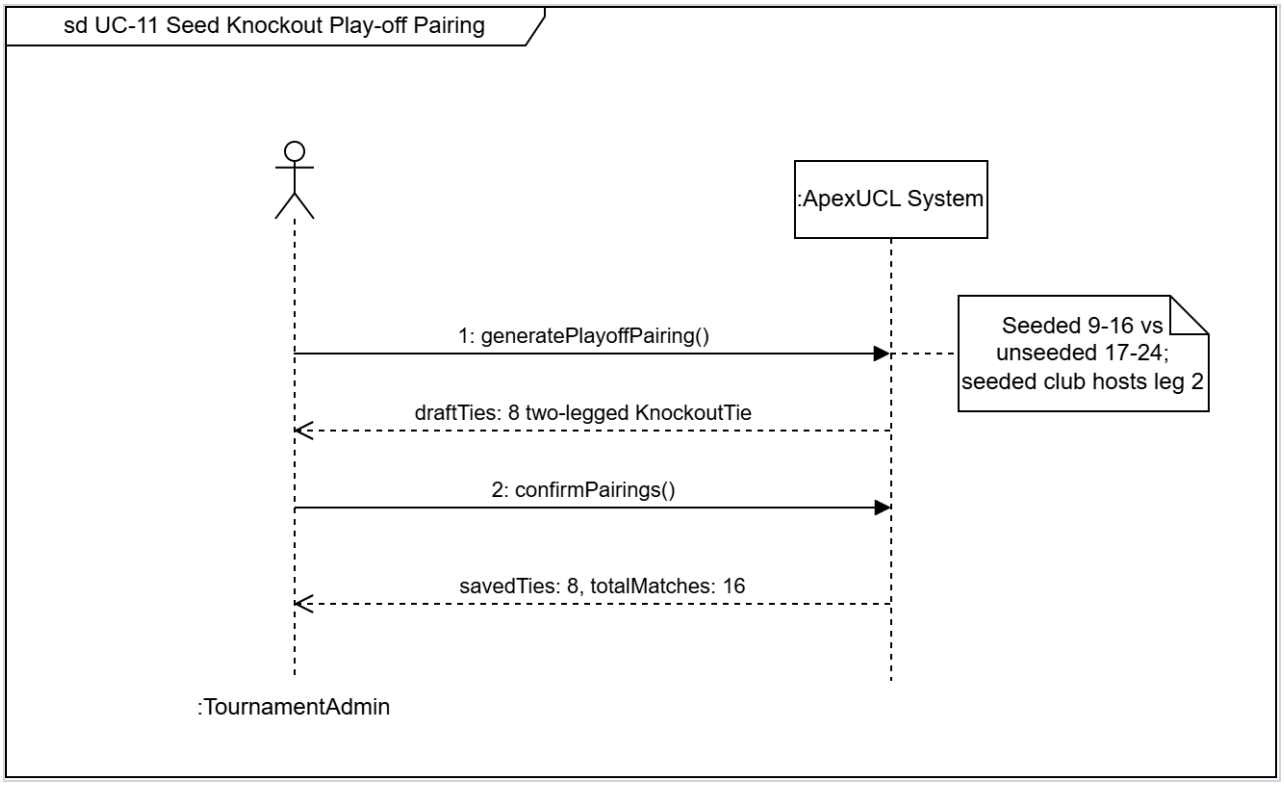
3.9 SSD UC-09: Manage Disciplinary Records & Suspensions



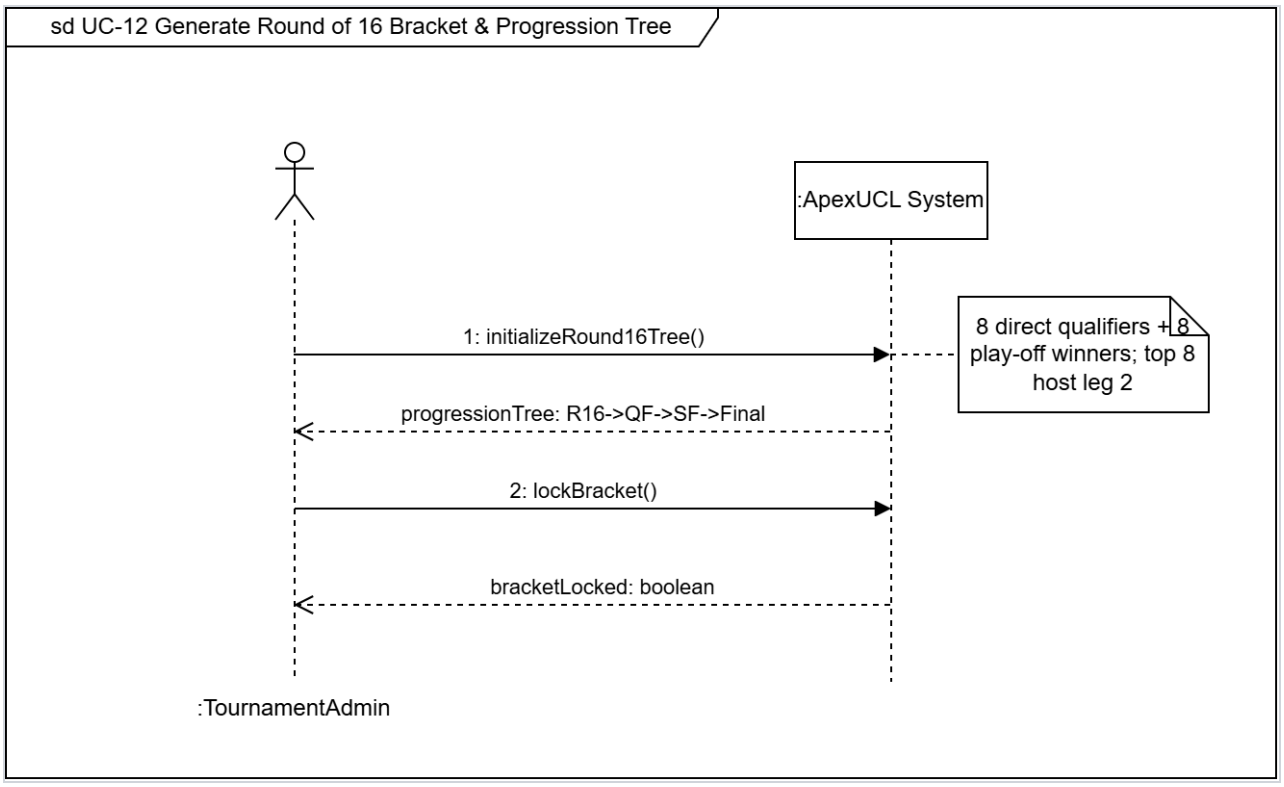
3.10 SSD UC-10: Partition Standings into Qualification Tiers



3.11 SSD UC-11: Seed Knockout Play-off Pairings



3.12 SSD UC-12: Generate Round of 16 Bracket & Progression Tree



4. Part Three: Code Integration

The Employee hierarchy is integrated into ApexUCL as a **Personnel & Payroll** module of the admin console (package `com.apexucl.personnel`), written in Java.

4.1 Salary rules

Category	Gross salary	Net salary
Faculty	salary + medical allowance + research allowance	gross − tax − pension
Staff	salary + medical allowance + bonus	gross − tax − pension
Research Assistant	salary + research fund + overtime	gross

Medical allowance is capped at 115,000 per year (Faculty and Staff) and bonus at 40,000 per year (Staff). Values above a cap are rejected and the user is asked again.

4.2 How override and polymorphism are managed

- **Inheritance:** `Faculty`, `Staff` and `ResearchAssistant` extend the abstract class `Employee`, which holds the common attributes (empNo, name, address, city, salary, jobTitle).
- **Overriding:** `Employee` declares abstract methods `getCategory()`, `calculateGrossSalary()`, `calculateNetSalary()`, `displayAllowances()` and `displayDeductions()`. Each subclass overrides them (`@Override`) with its own allowances and deductions.
- **Runtime polymorphism:** all employees are stored in one `List<Employee>`. The report calls `e.display()` and `e.calculateNetSalary()` on an `Employee` reference, and Java runs the subclass's version at runtime, with no type checks.
- **Overloading:** `display()` / `display(boolean)`, and two `ResearchAssistant` constructors (with and without overtime).

4.3 Source code

`Employee.java`

```java
package com.apexucl.personnel;

/**
 * Base class for every ApexUCL staff member on the payroll.
 *
 * Employee is abstract: nobody is "just an Employee", each person belongs to
 * exactly one category (Faculty, Staff or ResearchAssistant). The category-
 * specific rules for gross and net salary are declared here as abstract
 * methods and OVERRIDDEN in each subclass.
 */
public abstract class Employee {

    /** Shared rule for Faculty and Staff: medical allowance cap per year. */
    public static final double MAX_MEDICAL_ALLOWANCE = 115_000;

    private final int empNo;
    private final String name;
    private final String address;
    private final String city;
    private final double salary;
    private final String jobTitle;

    protected Employee(int empNo, String name, String address, String city,
                       double salary, String jobTitle) {
        if (salary < 0) {
            throw new IllegalArgumentException("Salary cannot be negative.");
        }
        this.empNo = empNo;
        this.name = name;
        this.address = address;
        this.city = city;
        this.salary = salary;
        this.jobTitle = jobTitle;
    }

    // ---- Methods every subclass MUST override (runtime polymorphism) ----

    /** Category label printed in the report, e.g. "Faculty". */
    public abstract String getCategory();

    /** Salary plus every allowance this category earns. */
    public abstract double calculateGrossSalary();

    /** Gross salary minus every deduction this category pays. */
    public abstract double calculateNetSalary();

    /** Prints the allowance lines that appear before Gross Salary. */
    protected abstract void displayAllowances();

    /** Prints the deduction lines that appear after Net Salary. */
    protected abstract void displayDeductions();

    // ---- Shared behaviour ----

    /**
     * Template for the report block. The common lines are printed here; the
     * category-specific lines come from the overridden hooks above, so the
     * same call prints a different block for each subclass.
     */
```

```
    public void display() {
        System.out.println("Employee name: " + name);
        System.out.println("Emp No: " + empNo);
        System.out.println("Job Title: " + jobTitle);
        System.out.println("City: " + city);
        printAmount("Salary", salary);
        displayAllowances();
        printAmount("Gross Salary", calculateGrossSalary());
        printAmount("Net Salary", calculateNetSalary());
        displayDeductions();
    }

    /** Overloaded display: optionally prints the category header first. */
    public void display(boolean withCategoryHeader) {
        if (withCategoryHeader) {
            System.out.println("Category: " + getCategory());
        }
        display();
    }

    protected static void printAmount(String label, double amount) {
        System.out.printf("%s: %,.0f%n", label, amount);
    }

    protected static double requireNonNegative(double value, String field) {
        if (value < 0) {
            throw new IllegalArgumentException(field + " cannot be negative.");
        }
        return value;
    }

    protected static double requireMedicalWithinCap(double medical) {
        requireNonNegative(medical, "Medical allowance");
        if (medical > MAX_MEDICAL_ALLOWANCE) {
            throw new IllegalArgumentException(String.format(
                    "Medical allowance %,.0f exceeds the yearly maximum of %,.0f.",
                    medical, MAX_MEDICAL_ALLOWANCE));
        }
        return medical;
    }

    public int getEmpNo() { return empNo; }
    public String getName() { return name; }
    public String getAddress() { return address; }
    public String getCity() { return city; }
    public double getSalary() { return salary; }
    public String getJobTitle() { return jobTitle; }
}
```

Faculty.java

```
package com.apexucl.personnel;

/**
 * Faculty: tax, medical allowance (max 115,000 per year), pension and
 * research allowance.
 *
 * Gross = salary + medical allowance + research allowance
 * Net   = gross - tax - pension
 */
public class Faculty extends Employee {

    private final double tax;
    private final double medicalAllowance;
    private final double pension;
```

```
    private final double researchAllowance;

    public Faculty(int empNo, String name, String address, String city,
                   double salary, String jobTitle,
                   double tax, double medicalAllowance,
                   double pension, double researchAllowance) {
        super(empNo, name, address, city, salary, jobTitle);
        this.tax = requireNonNegative(tax, "Tax");
        this.medicalAllowance = requireMedicalWithinCap(medicalAllowance);
        this.pension = requireNonNegative(pension, "Pension");
        this.researchAllowance = requireNonNegative(researchAllowance, "Research allowance");
    }

    @Override
    public String getCategory() {
        return "Faculty";
    }

    @Override
    public double calculateGrossSalary() {
        return getSalary() + medicalAllowance + researchAllowance;
    }

    @Override
    public double calculateNetSalary() {
        return calculateGrossSalary() - tax - pension;
    }

    @Override
    protected void displayAllowances() {
        printAmount("Medical Allowance", medicalAllowance);
        printAmount("Research Allowance", researchAllowance);
    }

    @Override
    protected void displayDeductions() {
        printAmount("Tax", tax);
        printAmount("Pension", pension);
    }

    public double getTax() { return tax; }
    public double getMedicalAllowance() { return medicalAllowance; }
    public double getPension() { return pension; }
    public double getResearchAllowance() { return researchAllowance; }
}
```

Staff.java

```
package com.apexucl.personnel;

/**
 * Staff: tax, medical allowance (max 115,000 per year), pension and
 * bonus (max 40,000 per year).
 *
 * Gross = salary + medical allowance + bonus
 * Net   = gross - tax - pension
 */
public class Staff extends Employee {

    public static final double MAX_BONUS = 40_000;

    private final double tax;
    private final double medicalAllowance;
    private final double pension;
    private final double bonus;
```

```
    public Staff(int empNo, String name, String address, String city,
                 double salary, String jobTitle,
                 double tax, double medicalAllowance,
                 double pension, double bonus) {
        super(empNo, name, address, city, salary, jobTitle);
        this.tax = requireNonNegative(tax, "Tax");
        this.medicalAllowance = requireMedicalWithinCap(medicalAllowance);
        this.pension = requireNonNegative(pension, "Pension");
        requireNonNegative(bonus, "Bonus");
        if (bonus > MAX_BONUS) {
            throw new IllegalArgumentException(String.format(
                    "Bonus %,.0f exceeds the yearly maximum of %,.0f.", bonus, MAX_BONUS));
        }
        this.bonus = bonus;
    }

    @Override
    public String getCategory() {
        return "Staff";
    }

    @Override
    public double calculateGrossSalary() {
        return getSalary() + medicalAllowance + bonus;
    }

    @Override
    public double calculateNetSalary() {
        return calculateGrossSalary() - tax - pension;
    }

    @Override
    protected void displayAllowances() {
        printAmount("Medical Allowance", medicalAllowance);
        printAmount("Bonus", bonus);
    }

    @Override
    protected void displayDeductions() {
        printAmount("Tax", tax);
        printAmount("Pension", pension);
    }

    public double getTax() { return tax; }
    public double getMedicalAllowance() { return medicalAllowance; }
    public double getPension() { return pension; }
    public double getBonus() { return bonus; }
}
```

ResearchAssistant.java

```
package com.apexucl.personnel;

/**
 * Research Assistant: research fund and overtime. The problem statement
 * gives Research Assistants no tax or pension, so net equals gross.
 *
 * Gross = salary + research fund + overtime
 * Net   = gross
 */
public class ResearchAssistant extends Employee {

    private final double researchFund;
    private final double overtime;
```

```
    public ResearchAssistant(int empNo, String name, String address, String city,
                             double salary, String jobTitle,
                             double researchFund, double overtime) {
        super(empNo, name, address, city, salary, jobTitle);
        this.researchFund = requireNonNegative(researchFund, "Research fund");
        this.overtime = requireNonNegative(overtime, "Overtime");
    }

    /** Overloaded constructor: a Research Assistant with no overtime yet. */
    public ResearchAssistant(int empNo, String name, String address, String city,
                             double salary, String jobTitle, double researchFund) {
        this(empNo, name, address, city, salary, jobTitle, researchFund, 0);
    }

    @Override
    public String getCategory() {
        return "Research Assistant";
    }

    @Override
    public double calculateGrossSalary() {
        return getSalary() + researchFund + overtime;
    }

    @Override
    public double calculateNetSalary() {
        return calculateGrossSalary();
    }

    @Override
    protected void displayAllowances() {
        printAmount("Research Fund", researchFund);
        printAmount("Overtime", overtime);
    }

    @Override
    protected void displayDeductions() {
        // No deductions defined for Research Assistants.
    }

    public double getResearchFund() { return researchFund; }
    public double getOvertime() { return overtime; }
}
```

PersonnelModule.java

```
package com.apexucl.personnel;

import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

/**
 * ApexUCL "Personnel & Payroll" module (Deliverable 4, Part 3).
 *
 * Every employee, whatever its category, is stored in ONE List<Employee>.
 * The report and the payroll totals never check the concrete type: they call
 * getCategory(), display(), calculateGrossSalary() and calculateNetSalary()
 * on an Employee reference and Java picks the overriding method at runtime.
 */
public class PersonnelModule {

    private static final String SEPARATOR = "-----------------------------------";
    private static final String[] CATEGORY_ORDER = {"Faculty", "Staff", "Research Assistant"};
```

```
    private final List<Employee> employees = new ArrayList<>();
    private final Scanner in;
    private int nextEmpNo = 1001;

    public PersonnelModule(Scanner in) {
        this.in = in;
    }

    /** Module menu shown from the ApexUCL admin console. */
    public void run() {
        while (true) {
            System.out.println();
            System.out.println("=== ApexUCL > Personnel & Payroll ===");
            System.out.println("1. Load sample personnel (4 Faculty, 2 Staff, 3 Research
Assistants)");
            System.out.println("2. Enter personnel manually");
            System.out.println("3. Show payroll report");
            System.out.println("0. Back to main menu");
            int choice = readInt("Choose: ", 0, 3);
            switch (choice) {
                case 1:
                    loadSampleData();
                    System.out.println(employees.size() + " employees on record.");
                    break;
                case 2:
                    enterEmployees();
                    break;
                case 3:
                    printReport();
                    break;
                case 0:
                    return;
                default:
                    break;
            }
        }
    }

    // ------------------------------------------------------------------ data

    public void addEmployee(Employee e) {
        employees.add(e); // upcast: a Faculty/Staff/RA stored as an Employee
    }

    public List<Employee> getEmployees() {
        return employees;
    }

    /** Sample set from the problem statement: 4 Faculty, 2 Staff, 3 Research Assistants. */
    public void loadSampleData() {
        employees.clear();
        nextEmpNo = 1001;
        addEmployee(new Faculty(nextEmpNo++, "Umar Ali", "House 12, Street 4, F-7/2", "Islamabad",
                450_000, "Professor", 40_000, 95_000, 10_000, 30_000));
        addEmployee(new Faculty(nextEmpNo++, "Muhammad Ahmad", "Flat 3, Block B, G-11/3",
"Islamabad",
                380_000, "Associate Professor", 32_000, 85_000, 9_000, 25_000));
        addEmployee(new Faculty(nextEmpNo++, "Ayesha Khan", "45 Mall Road", "Lahore",
                300_000, "Assistant Professor", 25_000, 70_000, 8_000, 20_000));
        addEmployee(new Faculty(nextEmpNo++, "Hassan Raza", "House 7, Jinnah Town", "Abbottabad",
                200_000, "Lecturer", 15_000, 60_000, 6_000, 10_000));

        addEmployee(new Staff(nextEmpNo++, "Bilal Ahmed", "House 22, Satellite Town", "Rawalpindi",
                150_000, "Fixture Operations Officer", 10_000, 50_000, 5_000, 40_000));
        addEmployee(new Staff(nextEmpNo++, "Sana Malik", "Plot 9, DHA Phase 2", "Karachi",
```

```
                120_000, "Administrative Officer", 8_000, 45_000, 4_000, 30_000));

        addEmployee(new ResearchAssistant(nextEmpNo++, "Zainab Tariq", "House 5, University Town",
"Peshawar",
                90_000, "Research Assistant (Tie-Breaker Analytics)", 25_000, 8_000));
        addEmployee(new ResearchAssistant(nextEmpNo++, "Usman Shah", "Street 10, Model Town",
"Lahore",
                85_000, "Research Assistant (Fixture Scheduling)", 20_000, 12_000));
        // Overloaded constructor: no overtime recorded yet.
        addEmployee(new ResearchAssistant(nextEmpNo++, "Fatima Noor", "House 31, I-8/4",
"Islamabad",
                80_000, "Research Assistant (Data Quality)", 18_000));
    }

    // ------------------------------------------------------------------ input

    private void enterEmployees() {
        int faculty = readInt("How many Faculty? ", 0, 100);
        int staff = readInt("How many Staff? ", 0, 100);
        int ra = readInt("How many Research Assistants? ", 0, 100);

        for (int i = 1; i <= faculty; i++) {
            System.out.println("\n-- Faculty " + i + " of " + faculty + " --");
            String[] c = readCommonFields();
            double salary = readAmount("Salary: ", Double.MAX_VALUE);
            double tax = readAmount("Tax: ", Double.MAX_VALUE);
            double medical = readAmount("Medical allowance (max 115,000): ",
Employee.MAX_MEDICAL_ALLOWANCE);
            double pension = readAmount("Pension: ", Double.MAX_VALUE);
            double research = readAmount("Research allowance: ", Double.MAX_VALUE);
            addEmployee(new Faculty(nextEmpNo++, c[0], c[1], c[2], salary, c[3],
                    tax, medical, pension, research));
        }
        for (int i = 1; i <= staff; i++) {
            System.out.println("\n-- Staff " + i + " of " + staff + " --");
            String[] c = readCommonFields();
            double salary = readAmount("Salary: ", Double.MAX_VALUE);
            double tax = readAmount("Tax: ", Double.MAX_VALUE);
            double medical = readAmount("Medical allowance (max 115,000): ",
Employee.MAX_MEDICAL_ALLOWANCE);
            double pension = readAmount("Pension: ", Double.MAX_VALUE);
            double bonus = readAmount("Bonus (max 40,000): ", Staff.MAX_BONUS);
            addEmployee(new Staff(nextEmpNo++, c[0], c[1], c[2], salary, c[3],
                    tax, medical, pension, bonus));
        }
        for (int i = 1; i <= ra; i++) {
            System.out.println("\n-- Research Assistant " + i + " of " + ra + " --");
            String[] c = readCommonFields();
            double salary = readAmount("Salary: ", Double.MAX_VALUE);
            double fund = readAmount("Research fund: ", Double.MAX_VALUE);
            double overtime = readAmount("Overtime: ", Double.MAX_VALUE);
            addEmployee(new ResearchAssistant(nextEmpNo++, c[0], c[1], c[2], salary, c[3],
                    fund, overtime));
        }
        System.out.println(employees.size() + " employees on record.");
    }

    /** Returns {name, address, city, jobTitle}. */
    private String[] readCommonFields() {
        String name = readText("Name: ");
        String address = readText("Address: ");
        String city = readText("City: ");
        String jobTitle = readText("Job title: ");
        return new String[] {name, address, city, jobTitle};
    }
```

```
    private String readText(String prompt) {
        while (true) {
            System.out.print(prompt);
            String line = in.nextLine().trim();
            if (!line.isEmpty()) {
                return line;
            }
            System.out.println("  Please enter a value.");
        }
    }

    private int readInt(String prompt, int min, int max) {
        while (true) {
            System.out.print(prompt);
            String line = in.nextLine().trim();
            try {
                int value = Integer.parseInt(line);
                if (value >= min && value <= max) {
                    return value;
                }
            } catch (NumberFormatException ignored) {
                // fall through to the message below
            }
            System.out.println("  Enter a whole number from " + min + " to " + max + ".");
        }
    }

    private double readAmount(String prompt, double max) {
        while (true) {
            System.out.print(prompt);
            String line = in.nextLine().trim().replace(",", "");
            try {
                double value = Double.parseDouble(line);
                if (value < 0) {
                    System.out.println("  Amount cannot be negative.");
                } else if (value > max) {
                    System.out.printf("  Amount cannot exceed %,.0f per year.%n", max);
                } else {
                    return value;
                }
            } catch (NumberFormatException e) {
                System.out.println("  Enter a number, e.g. 95000 or 95,000.");
            }
        }
    }

    // ---------------------------------------------------------------- report

    /** Prints every employee grouped by category, then polymorphic payroll totals. */
    public void printReport() {
        if (employees.isEmpty()) {
            System.out.println("No employees on record. Load the sample or enter personnel first.");
            return;
        }
        System.out.println();
        for (String category : CATEGORY_ORDER) {
            boolean headerPrinted = false;
            for (Employee e : employees) {
                if (!e.getCategory().equals(category)) {
                    continue;
                }
                if (!headerPrinted) {
                    System.out.println("Category: " + category);
                    headerPrinted = true;
                }
                e.display();              // dynamic dispatch picks the subclass's version
```

```
                System.out.println(SEPARATOR);
            }
        }

        double totalGross = 0;
        double totalNet = 0;
        for (Employee e : employees) {   // same loop, no instanceof, three different rules
            totalGross += e.calculateGrossSalary();
            totalNet += e.calculateNetSalary();
        }
        System.out.println("Payroll summary (" + employees.size() + " employees)");
        System.out.printf("Total Gross Salary: %,.0f%n", totalGross);
        System.out.printf("Total Net Salary: %,.0f%n", totalNet);
    }
}
```

ApexUCLApp.java

```
package com.apexucl;

import com.apexucl.personnel.PersonnelModule;

import java.util.Scanner;

/**
 * ApexUCL admin console entry point.
 *
 * Run with --sample to print the Part 3 payroll report straight away:
 *   java -cp out com.apexucl.ApexUCLApp --sample
 */
public class ApexUCLApp {

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        PersonnelModule personnel = new PersonnelModule(in);

        if (args.length > 0 && args[0].equals("--sample")) {
            personnel.loadSampleData();
            personnel.printReport();
            return;
        }

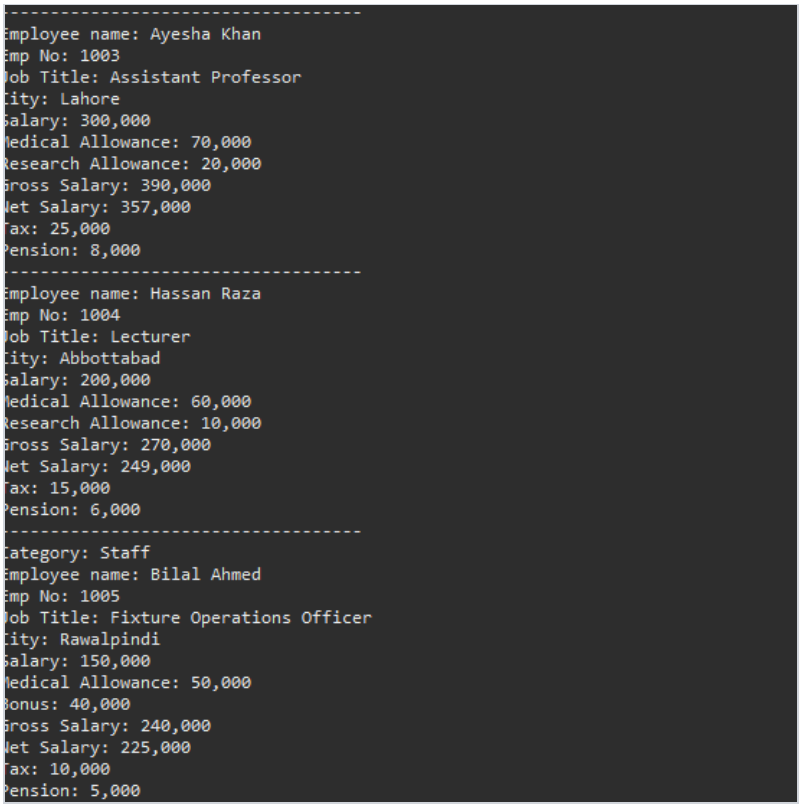
        while (true) {
            System.out.println();
            System.out.println("========== ApexUCL Admin Console ==========");
            System.out.println("1. Personnel & Payroll");
            System.out.println("0. Exit");
            System.out.print("Choose: ");
            if (!in.hasNextLine()) {
                return;
            }
            String choice = in.nextLine().trim();
            if (choice.equals("1")) {
                personnel.run();
            } else if (choice.equals("0")) {
                System.out.println("Goodbye.");
                return;
            } else {
                System.out.println("Enter 1 or 0.");
            }
        }
    }
}
```

4.4 Program output: sample input

Sample run with 4 Faculty, 2 Staff and 3 Research Assistants. The input was supplied from `sample_input.txt` (Eclipse Run Configuration → Input File), so the entered values are not echoed on the console; the report below shows them.

```
========== ApexUCL Admin Console ==========
1. Personnel & Payroll
0. Exit
Choose:
=== ApexUCL > Personnel & Payroll ===
1. Load sample personnel (4 Faculty, 2 Staff, 3 Research Assistants)
2. Enter personnel manually
3. Show payroll report
0. Back to main menu
Choose: How many Faculty? How many Staff? How many Research Assistants?
-- Faculty 1 of 4 --
Name: Address: City: Job title: Salary: Tax: Medical allowance (max 115,000): Pension: Research allowance:
-- Faculty 2 of 4 --
Name: Address: City: Job title: Salary: Tax: Medical allowance (max 115,000): Pension: Research allowance:
-- Faculty 3 of 4 --
Name: Address: City: Job title: Salary: Tax: Medical allowance (max 115,000): Pension: Research allowance:
-- Faculty 4 of 4 --
Name: Address: City: Job title: Salary: Tax: Medical allowance (max 115,000): Pension: Research allowance:
-- Staff 1 of 2 --
Name: Address: City: Job title: Salary: Tax: Medical allowance (max 115,000): Pension: Bonus (max 40,000):
-- Staff 2 of 2 --
Name: Address: City: Job title: Salary: Tax: Medical allowance (max 115,000): Pension: Bonus (max 40,000):
-- Research Assistant 1 of 3 --
Name: Address: City: Job title: Salary: Research fund: Overtime:
-- Research Assistant 2 of 3 --
Name: Address: City: Job title: Salary: Research fund: Overtime:
-- Research Assistant 3 of 3 --
Name: Address: City: Job title: Salary: Research fund: Overtime: 9 employees on record.
```

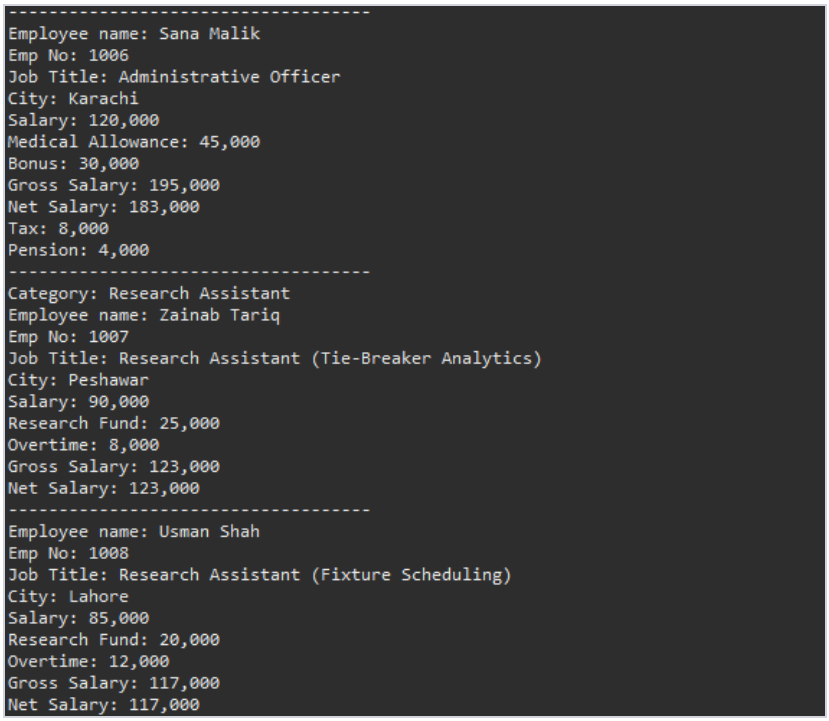
```
=== ApexUCL > Personnel & Payroll ===
1. Load sample personnel (4 Faculty, 2 Staff, 3 Research Assistants)
2. Enter personnel manually
3. Show payroll report
0. Back to main menu
Choose:
Category: Faculty
Employee name: Umar Ali
Emp No: 1001
Job Title: Professor
City: Islamabad
Salary: 450,000
Medical Allowance: 95,000
Research Allowance: 30,000
Gross Salary: 575,000
Net Salary: 525,000
Tax: 40,000
Pension: 10,000
-----------------------------------
Employee name: Muhammad Ahmad
Emp No: 1002
Job Title: Associate Professor
City: Islamabad
Salary: 380,000
Medical Allowance: 85,000
Research Allowance: 25,000
Gross Salary: 490,000
Net Salary: 449,000
Tax: 32,000
Pension: 9,000
```



Employee name: Ayesha Khan
Emp No: 1003
Job Title: Assistant Professor
City: Lahore
Salary: 300,000
Medical Allowance: 70,000
Research Allowance: 20,000
Gross Salary: 390,000
Net Salary: 357,000
Tax: 25,000
Pension: 8,000

Employee name: Hassan Raza
Emp No: 1004
Job Title: Lecturer
City: Abbottabad
Salary: 200,000
Medical Allowance: 60,000
Research Allowance: 10,000
Gross Salary: 270,000
Net Salary: 249,000
Tax: 15,000
Pension: 6,000

Category: Staff
Employee name: Bilal Ahmed
Emp No: 1005
Job Title: Fixture Operations Officer
City: Rawalpindi
Salary: 150,000
Medical Allowance: 50,000
Bonus: 40,000
Gross Salary: 240,000
Net Salary: 225,000
Tax: 10,000
Pension: 5,000



Employee name: Sana Malik
Emp No: 1006
Job Title: Administrative Officer
City: Karachi
Salary: 120,000
Medical Allowance: 45,000
Bonus: 30,000
Gross Salary: 195,000
Net Salary: 183,000
Tax: 8,000
Pension: 4,000

Category: Research Assistant
Employee name: Zainab Tariq
Emp No: 1007
Job Title: Research Assistant (Tie-Breaker Analytics)
City: Peshawar
Salary: 90,000
Research Fund: 25,000
Overtime: 8,000
Gross Salary: 123,000
Net Salary: 123,000

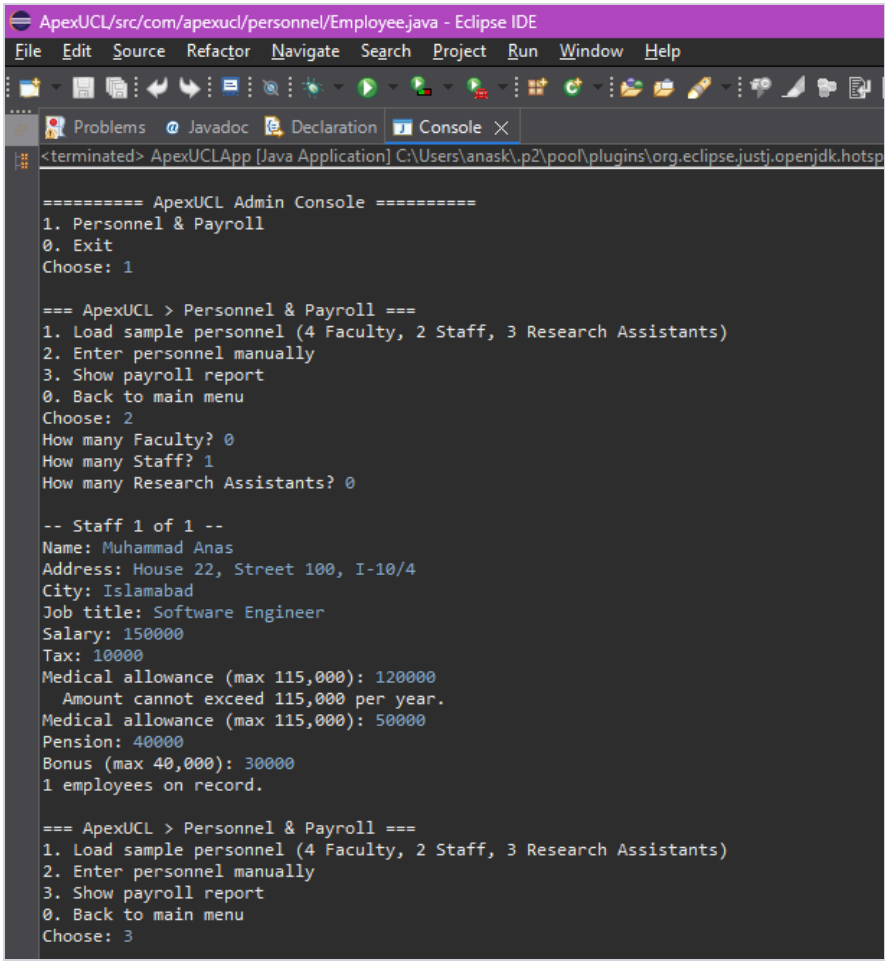
Employee name: Usman Shah
Emp No: 1008
Job Title: Research Assistant (Fixture Scheduling)
City: Lahore
Salary: 85,000
Research Fund: 20,000
Overtime: 12,000
Gross Salary: 117,000
Net Salary: 117,000

```
-----------------------------------
Employee name: Fatima Noor
Emp No: 1009
Job Title: Research Assistant (Data Quality)
City: Islamabad
Salary: 80,000
Research Fund: 18,000
Overtime: 0
Gross Salary: 98,000
Net Salary: 98,000
-----------------------------------
Payroll summary (9 employees)
Total Gross Salary: 2,498,000
Total Net Salary: 2,326,000

=== ApexUCL > Personnel & Payroll ===
1. Load sample personnel (4 Faculty, 2 Staff, 3 Research Assistants)
2. Enter personnel manually
3. Show payroll report
0. Back to main menu
Choose:
========== ApexUCL Admin Console ==========
1. Personnel & Payroll
0. Exit
Choose: Goodbye.
```

4.5 Program output: manual input and allowance cap check

One Staff member entered from the keyboard. A medical allowance of 120,000 is rejected because it exceeds the 115,000 yearly maximum, and the user is asked again.



```
Category: Staff
Employee name: Muhammad Anas
Emp No: 1001
Job Title: Software Engineer
City: Islamabad
Salary: 150,000
Medical Allowance: 50,000
Bonus: 30,000
Gross Salary: 230,000
Net Salary: 180,000
Tax: 10,000
Pension: 40,000
-----------------------------------
Payroll summary (1 employees)
Total Gross Salary: 230,000
Total Net Salary: 180,000

=== ApexUCL > Personnel & Payroll ===
1. Load sample personnel (4 Faculty, 2 Staff, 3 Research Assistants)
2. Enter personnel manually
3. Show payroll report
0. Back to main menu
Choose: 0

========== ApexUCL Admin Console ==========
1. Personnel & Payroll
0. Exit
Choose: 0
Goodbye.
```